

OpenModelica: A Powerful Engineering Modelling and Simulation Tool

The use of modelling and simulation in engineering has long been an accepted practice. Models represent the real systems either as physical reproductions in a smaller scale, or as mathematical and computer models that allow representing the dynamics of the system via simulation. Models enable you to study a system's behaviour in ways that are either not possible or too risky in the real world.

Modelling is at the heart of modern engineering practice. Engineers model things all the time. Models allow engineers to conduct many 'what if' experiments and also to study potential failures in the system. While mathematical models have been in use for a long time, in the last few decades, computer models have become indispensable. But one problem remains — the different ways in which engineers from different streams build the models. For example, mechanical engineers usually use CAD models and Finite Element Analysis. Electrical engineers use circuit diagrams, while a computer science engineer may use flow charts and object-oriented modelling in programming.

There have been attempts to bridge this gap, though. One particular solution is the Modelica language, which is for engineering design and simulation. Modelica is a domain neutral language with which small, simple things like resistors, as well as large and complex systems like automobiles and power plants can be modelled.

The Modelica language

Modelica is an object-oriented, equation based modelling language. It is not proprietary, and is managed by the Modelica Association. The purpose of Modelica is to describe engineering components and systems. We cannot think of Modelica as a programming language such as C++ or Java. Modelica needs an implementation environment. It is important to remember this — Modelica is a modelling language. Once we have modelled an engineering system, we can run the model and see how the system behaves. For example, if you've built a model of a braking system for a car, you can run the simulation and see how the braking system behaves under different load conditions. Or if you have built a circuit model, when you simulate the model, you can see how current and voltage change with respect to time

in different components. Modelica model files have .mo as the extension.

As mentioned earlier, we'll need to implement an environment for Modelica models to run. There are many such run environments. Some are open source, and some are proprietary. Important run environments for Modelica are OpenModelica, JModelica and Dymola. While OpenModelica and JModelica are free and open source platforms, Dymola is a commercial platform. Personally I would prefer OpenModelica as it is cross-platform, easy to use/install, comes with many other supporting tools, has very good documentation for learning, and has a reasonably good community around it, which is very essential.

Modelica's components library

Modelica's components library is a very powerful feature. As Modelica is an object-oriented modelling language, the idea of re-using components is very naturally exploited. And to make things even more interesting, Modelica Association releases what's in the components library on a free basis. The library is basically a collection of models of the most commonly used engineering components. To be more specific, it contains Blocks (mathematical/logical function blocks), ComplexBlocks, StateGraph, Electrical, Magnetic, Mechanics, Fluid, Media, Thermal, Math, ComplexMath, Utilities, Constants, Icons, and Slunits packages. Each of these 'packages' contain different component models under different sub-sections. Now imagine that you're building a model of a robotic arm, which contains both electrical and mechanical components, along with control software. You don't have to build models of components like the motor or shaft. You can just use the available component models in the library, fine tune the model parameters and 'connect' to see how your system works.

